```
    Float(@total_score) / @count
  end
end

scores = ScoreKeeper.new
scores << 10 << 20 << 40
puts "Average = #{scores.average}"
```

*produces:*

```
Average = 23.333333333333332
```

Note that there's a subtlety in this code—the << method explicitly returns self. It does this to allow the method chaining in the line scores << 10 << 20 << 40. Because each call to << returns the scores object, you can then call << again, passing in a new score. (Arrays also implement << the same way, and for the same reason.)

In addition to operators such as +, *, and <<, indexing using square brackets is also implemented as a method call. When you write this:

```
some_obj[1, 2, 3]
```

you're actually calling a method named [] on some_obj, passing it three parameters, equivalent to some_obj.[](1, 2, 3). You'd define this method using this:

```
class SomeClass
  def [](p1, p2, p3)
    # ...
  end
end
```

Similarly, assignment to an element is implemented using the []= method. This method receives each object passed as an index as its first *n* parameters and the value of the assignment as its last parameter:

```
class SomeClass
  def []=(*params)
    value = params.pop
    puts "Indexed with #{params.join(', ')}"
    puts "value = #{value.inspect}"
  end
end

s = SomeClass.new
s[1] = 2
s['cat', 'dog'] = 'enemies'
```

*produces:*

```
Indexed with 1
value = 2
Indexed with cat, dog
value = "enemies"
```

## Command Expressions

If you enclose a string in backquotes (sometimes called *backticks*) or use the delimited form %x{...}, the string will (by default) be executed as a command by your underlying operating system. The value returned is the standard output of that command. Newlines will not be